Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026

Interface graphique (GUI)

Introduction

- Les interfaces graphiques utilisateur (**GUI** – Graphical User Interfaces) permettent d'interagir avec un système à travers des icônes, des boutons, des menus, et divers éléments visuels, contrairement aux interfaces textuelles qui nécessitent de taper des commandes au clavier.
- Dans une **GUI**, **chaque composant graphique** — ainsi que l'interface dans son ensemble — **est associé à une ou plusieurs routines utilisateur**, appelées **callbacks** (ou rappels). L'exécution d'un callback est déclenchée par une action de l'utilisateur, comme : un clic de souris, la sélection d'un élément de menu, un déplacement de la souris sur un composant, la saisie au clavier.
- Ce paradigme correspond au modèle de **programmation événementielle** (**event-driven programming**), dans lequel l'application réagit aux événements générés par l'utilisateur ou le système.

Interface graphique (GUI)

Introduction

- La programmation conduite par les événements est **un paradigme imposé par les interfaces graphiques (GUI)**.
- Les actions de l'utilisateur — déplacement de la souris, clic, pression d'une touche, sélection d'un élément, etc. — génèrent des **événements** qui sont placés dans une **file d'attente**.
- Le programme récupère ensuite ces événements **un par un** et les traite en exécutant les **callbacks** (routines associées) correspondants. Ainsi, le flux d'exécution du programme n'est plus linéaire : il dépend entièrement des événements produits par l'utilisateur ou le système.

Interface graphique (GUI)

API utilisées pour les interfaces graphiques en Java

1. **AWT** (Abstract Window Toolkit) – JDK 1.1

- Première bibliothèque graphique fournie par Java.
- Basée sur les composants “lourds” (heavyweight), dépendants du système d’exploitation.
- Offre les bases : fenêtres, boutons, menus…
- Toujours présente mais rarement utilisée seule aujourd’hui.

2. **Swing** – JDK 1.2

- Construit au-dessus de AWT.
- Introduit des composants “légers” (lightweight), indépendants du système d’exploitation.
- Plus riche en fonctionnalités : tableaux (JTable), onglets (JTabbedPane), dialogues (JOptionPane), etc.
- Hautement personnalisable (look-and-feel, rendering…).

Interface graphique (GUI)

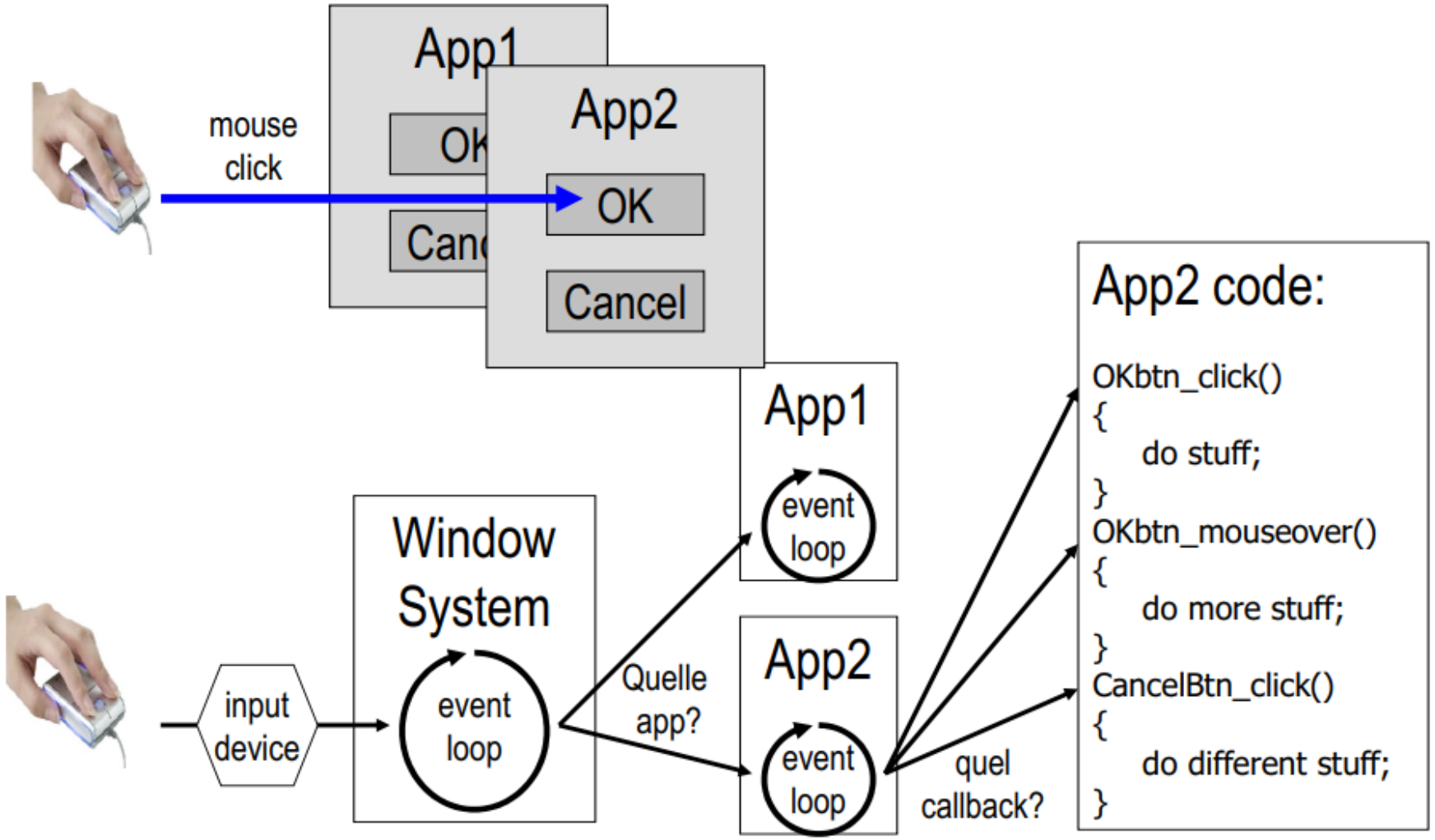
API utilisées pour les interfaces graphiques en Java

3. AWT + Swing = **JFC** (Java Foundation Classes)

- Swing et AWT font partie du JFC, l'ensemble des bibliothèques Java pour créer des interfaces graphiques.
- JFC inclut :
  - AWT
  - Swing
  - Java 2D API
  - Accessibilité
  - Drag and Drop

Interface graphique (GUI)

Évènements d'une GUI



Interface graphique (GUI)

Les écouteurs

- Swing utilise un Pattern de type :

  « observateur - observé »

- Les composants graphiques (boutons, listes, menus,…) sont les observés

- Chaque composant graphique a ses observateurs (ou écouteurs, listeners) qui s'enregistrent auprès de lui comme écouteur (ou observateur) d'un certain type d'événement

Interface graphique (GUI)

Les écouteurs

- Lorsqu'un événement survient sur un composant graphique, **l'écouteur** (listener) qui lui est associé en est automatiquement informé. Il exécute alors le **callback** correspondant à cet événement.
- Par exemple, l'écouteur associé au bouton « Exit » peut :
  - afficher une fenêtre de confirmation à l'utilisateur,
  - puis terminer l'application si l'utilisateur valide sa décision.

Interface graphique (GUI)

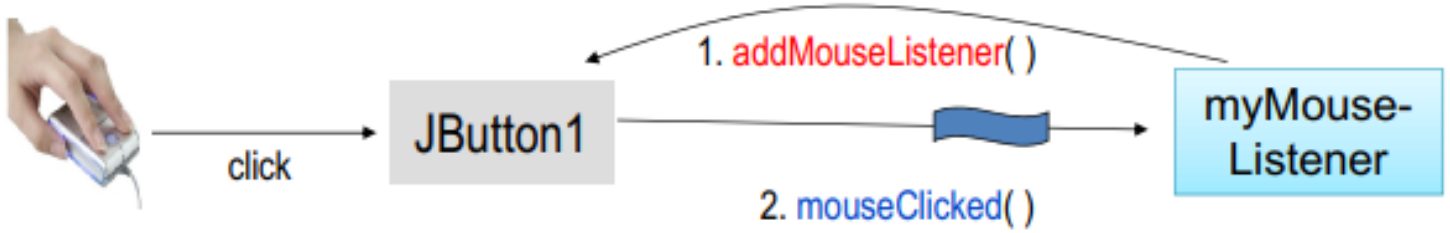
Les écouteurs : Exemple

1. Enregistrer l'écouteur auprès d'une composante source d'événement
   - Donner à la composante une référence de l'objet d'écoute

     jbutton1.addMouseListener(new myMouseListener());

2. Recevoir des événements de la composante
   - La composante appelle le callback sur l'objet Listener

     myMouseListener.mouseClicked(event)



Les écouteurs : Exemple

- Un listener (écouteur) d'événement Window permet, par exemple, de fermer l'application lorsque l'on ferme la fenêtre principale.
- Exemple :

```
import java.awt.event.*;
public class MonListener implements WindowListener {
   public void windowClosing(WindowEvent e) { System.exit(0); }
// Attention : On doit implémenter  toutes les méthodes de l'interface!!
   public void windowOpened(WindowEvent e) {…}
   public void windowClosed(WindowEvent e) {…}
   public void windowIconified(WindowEvent e) {…}
   public void windowDeiconified(WindowEvent e) {…}
   public void windowActivated(WindowEvent e) {…}
   public void windowDeactivated(WindowEvent e) {…}
}
```

Interface graphique (GUI)

Les écouteurs : Exemple

- Sur les boutons les événement permettent l'exécution d'un traitement si un click a eu lieux.
- Exemple :

```
public class MonListener implements java.awt.event.ActionListener {
public void actionPerformed(ActionEvent e) {
                System.out.println("le bouton a été clické");
    }
}
```

Dans le programme :

```
JButton b = new JButton("Click Me !");
b.addActionListener(new MonListener());
```

Interface graphique (GUI)

Architecture Swing

- **JComponent** est la classe de base de tous les composants Swing.
- Tous les composants Swing (**JButton, JLabel, JTextField**, etc.) héritent de **JComponent**.
- Fonctionnalités principales de **JComponent** :
  - **ToolTips** : textes d'aide affichés au survol du composant.
  - **Bordures** (borders) : permet de définir l'aspect visuel du contour du composant.
- **JComponent** représente **l'entité graphique la plus abstraite dans Swing** : toutes les fonctionnalités communes aux composants Swing y sont définies.

Interface graphique (GUI)

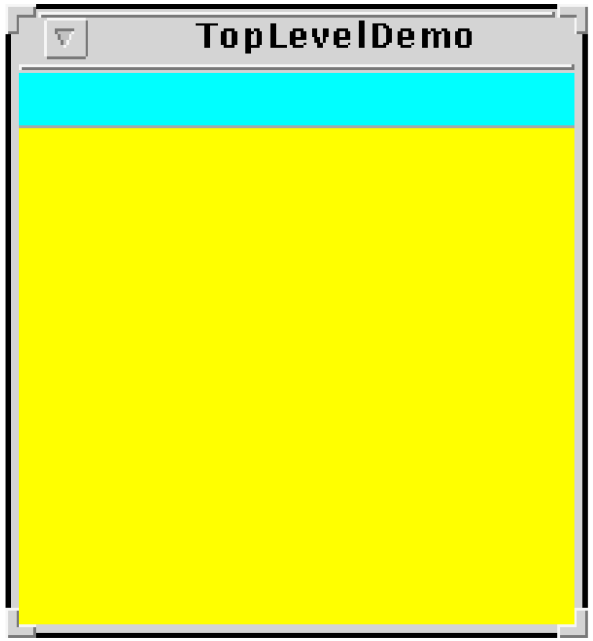
Top-Level

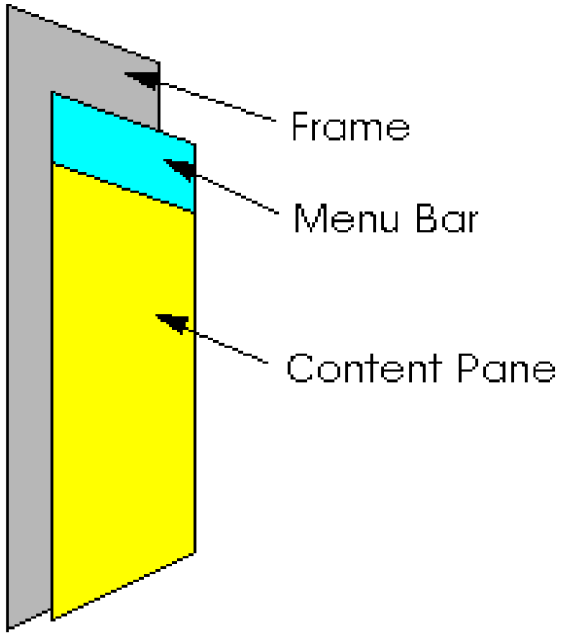
- Swing propose 3 composants top-level:
  - **JFrame**,
  - **JDialog**
  - **JApplet**
- Une application graphique doit avoir un composant top-level comme composant racine (composant qui inclus tous les autres composants)

Interface graphique (GUI)

Top-Level

- Les composants top-level possèdent un contentPane qui contient tous les composants visibles d'un top-level
- Un composant top-level peut contenir une barre de menu

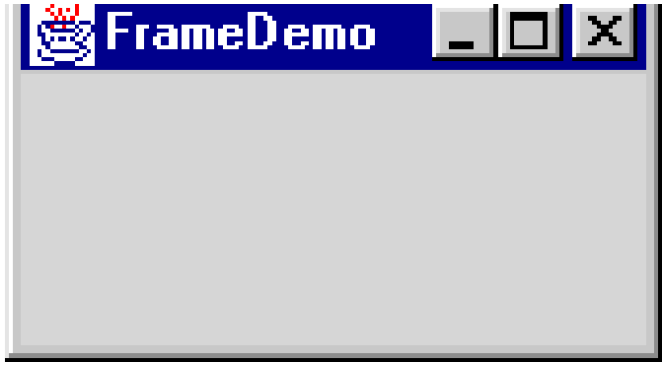




Interface graphique (GUI)

JFrame

- Une JFrame est une fenêtre avec un titre et une bordure



- Quelques méthodes :
  - ***public JFrame()***
  - ***public JFrame(String name)***
  - ***public Container getContentPane()***
  - ***public void setJMenuBar(JMenuBar menu)***
  - ***public void setTitle(String title)***
  - ***public void setIconImage(Image image)***

Interface graphique (GUI)

JDialog

- Un JDialog est une fenêtre qui a pour but un échange d'information
- Un JDialog dépend d'une fenêtre, si celle-ci est détruite, le JDialog l'est aussi
- Un JDilaog peut aussi être modal

Interface graphique (GUI)

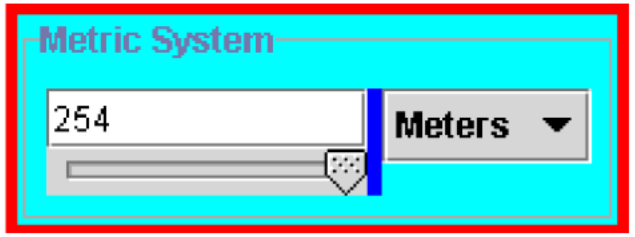
Conteneur Intermédiaire

- Les conteneur intermédiaire sont utilisés pour structurer l'application graphique
- Le composant top-level contient des composants conteneur intermédiaire
- Un conteneur intermédiaire peut contenir d'autre conteneurs intermédiaire
- Swing propose plusieurs conteneurs intermédiaire:
  - JPanel
  - JScrollPane
  - JSplitPane
  - JTabbedPane
  - JToolBar
  - …

Interface graphique (GUI)

JPanel

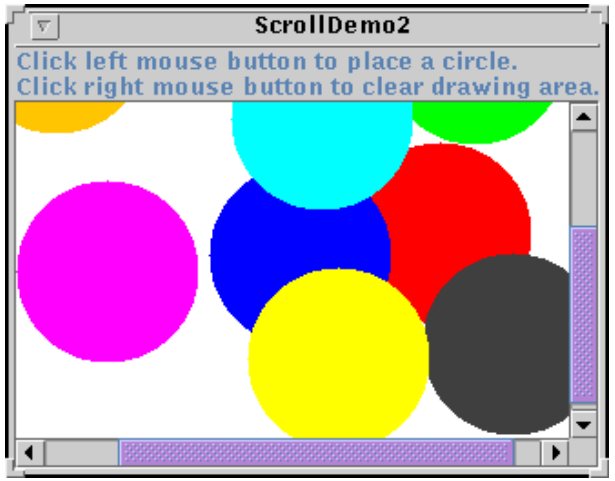
- Le JPanel est le conteneur intermédiaire le plus neutre
- On ne peut que choisir la couleur de fond



- Quelques méthodes de JPanel:
  - ***public JPanel()***
  - ***public Void add(Component comp)***
  - ***public void setLayout(LayoutManager lm)***

JScrollPane

- Un JScrollPane est un conteneur qui offre des ascenseurs, il permet de visionner un composant plus grand que lui

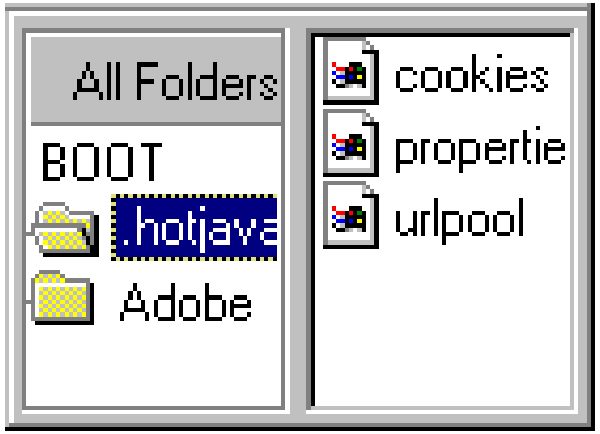


- Quelques méthodes:
  - **public JScrollPane(Component comp)**
  - **public void setCorner(String key,Component comp)**

Interface graphique (GUI)

JSplitPane

- Un JSplitPane est un panel coupé en deux par une barre de séparation. Un JSplitPane accueil deux composants.

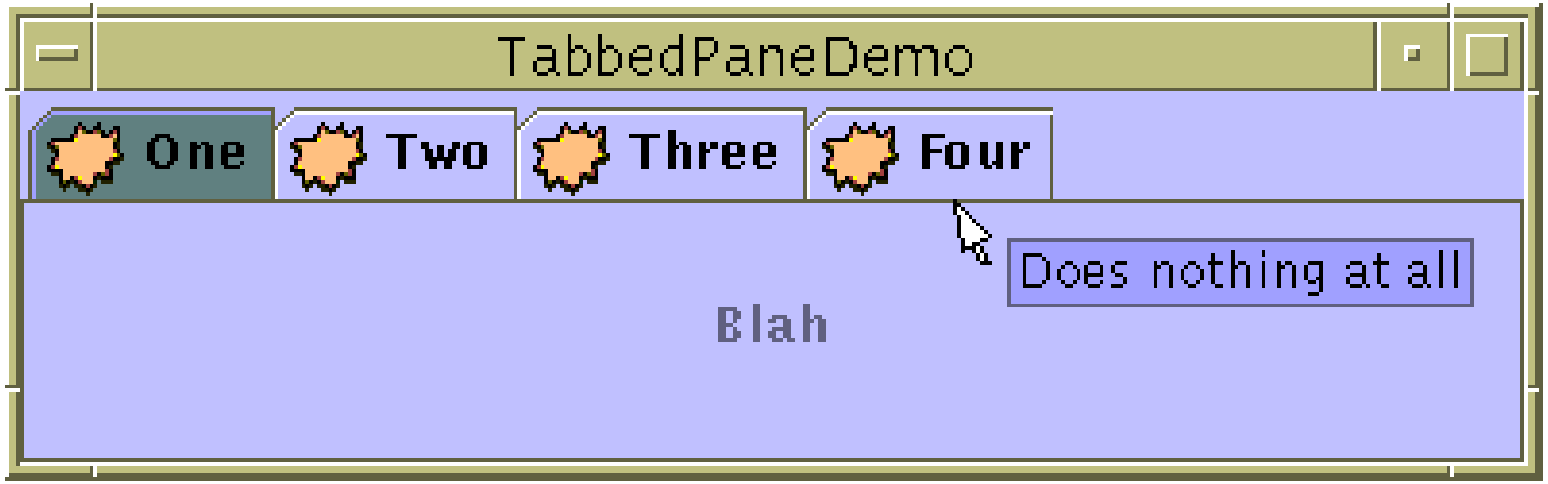


- Quelques méthodes:
  - **public JSplitPane(int ori, Component comp, Component c)**
  - **public void setDividerLocation(double pourc)**

Interface graphique (GUI)

JTabbedPane

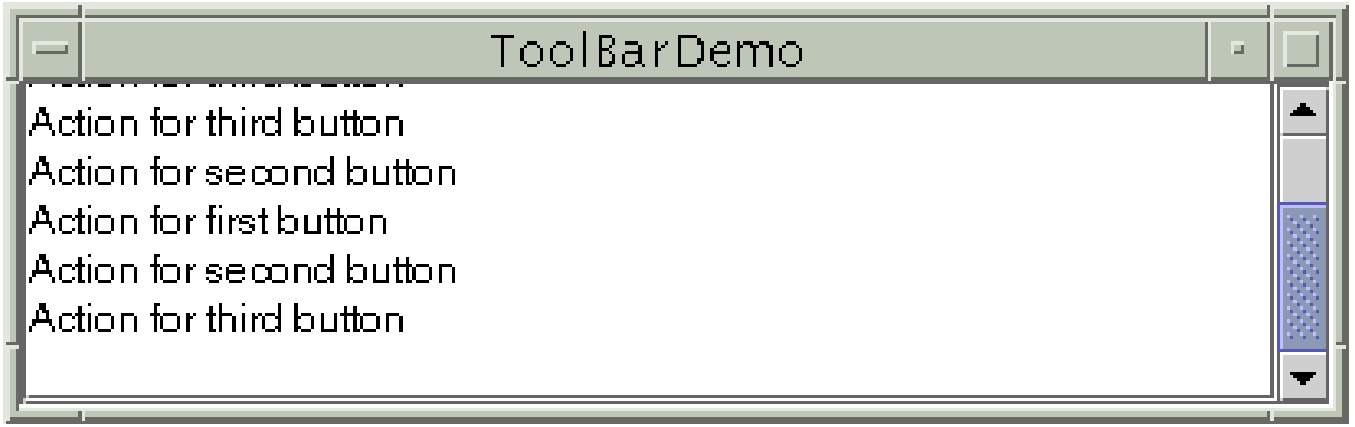
- Un JTabbedPane permet d'avoir des onglets

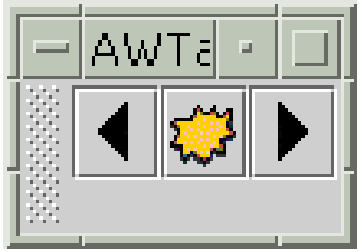


- Quelques méthodes:
  - **public JTabbedPane()**
  - **public void addTab(String s, Icon i, Component c, String s)**

JToolBar

- Une JToolBar est une barre de menu





- Quelques méthodes:
  - ***public JToolBar()***
  - ***public Component add(Component c)***

Interface graphique (GUI)

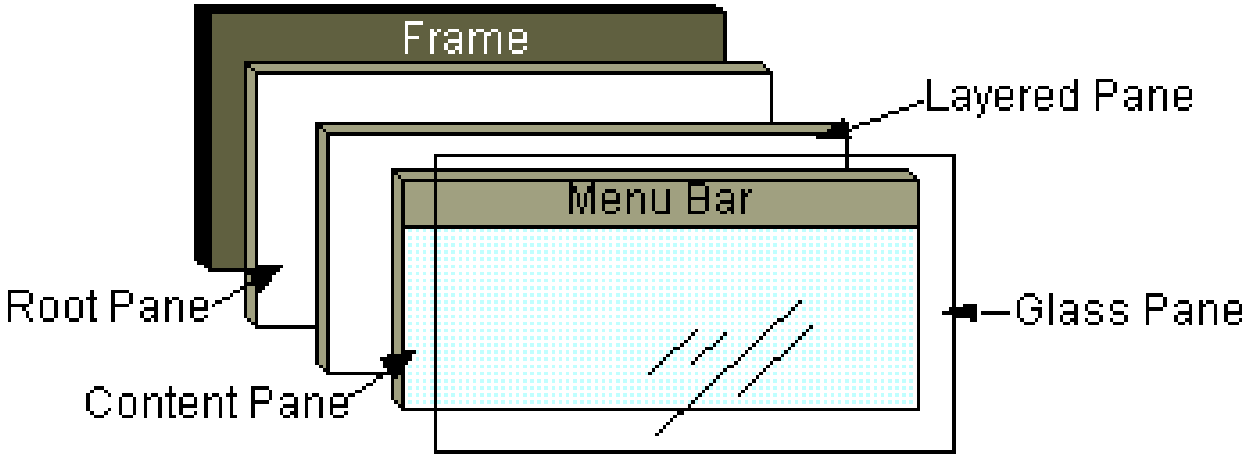
Conteneur Intermédiaire Spécialisé

- Les conteneur Intermédiaire spécialisé sont des conteneurs qui offrent des propriétés particulières aux composants qu'ils accueillent
  - JRootPane
  - JLayeredPane
  - JInternalFrame

Interface graphique (GUI)

JRootPane

- En principe, un JRootPane est obtenu à partir d'un top-level ou d'une JInternalFrame
- Un JRootPane est composé de :
  - glass pane
  - layered pane
  - content pane
  - menu bar



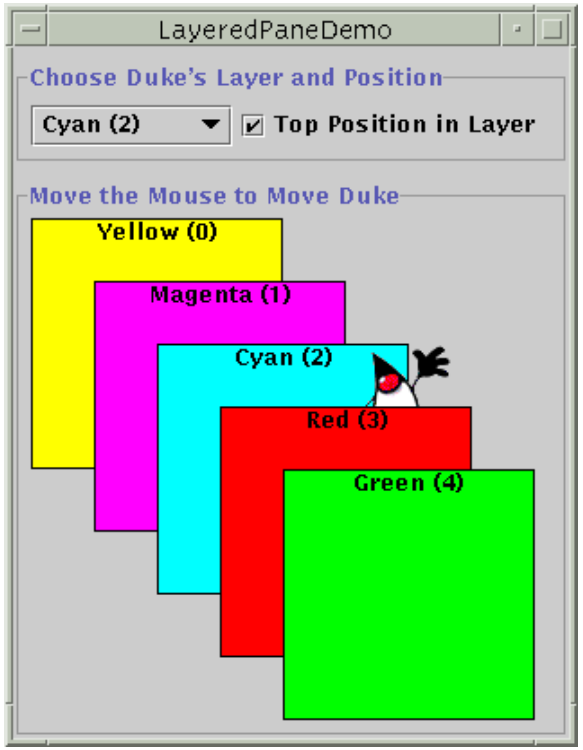
Interface graphique (GUI)

JLayeredPane

- Un JLayeredPane permet de positionner les composants dans un espace à trois dimensions

- Pour ajouter un Composant:

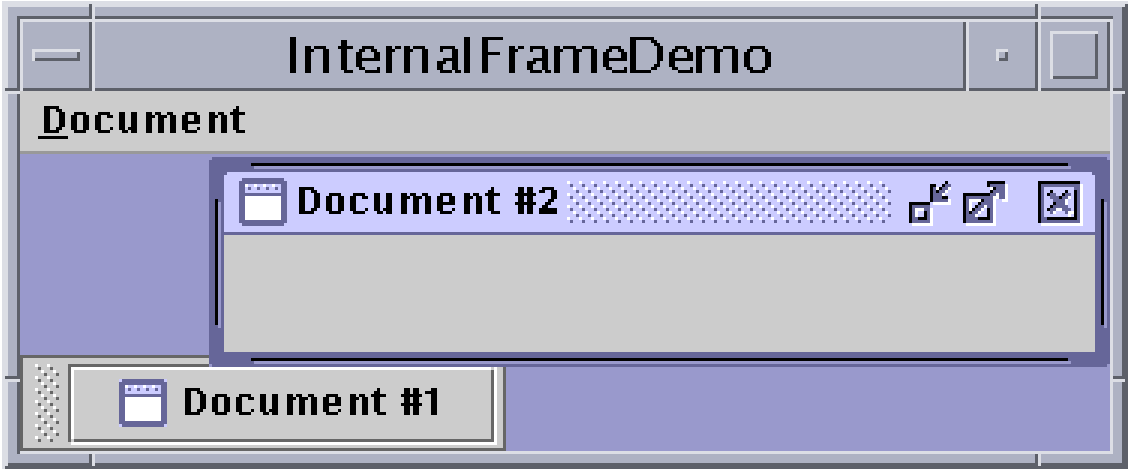
- ***add(Component c, Integer i)***



Interface graphique (GUI)

JInternaleFrame

- Un JInternaleFrame permet d'avoir des petites fenêtres dans une fenêtre.
- Une JInternaleFrame ressemble très fortement à une JFrame mais ce n'est pas un container Top-Level



Interface graphique (GUI)

Les composants atomiques

- Un composant atomique est considéré comme étant une entité unique.
- Java propose beaucoup de composants atomiques:
  - boutons, CheckBox, Radio
  - Combo box
  - List, menu
  - TextField, TextArea, Label
  - FileChooser, ColorChooser,
  - …

Interface graphique (GUI)

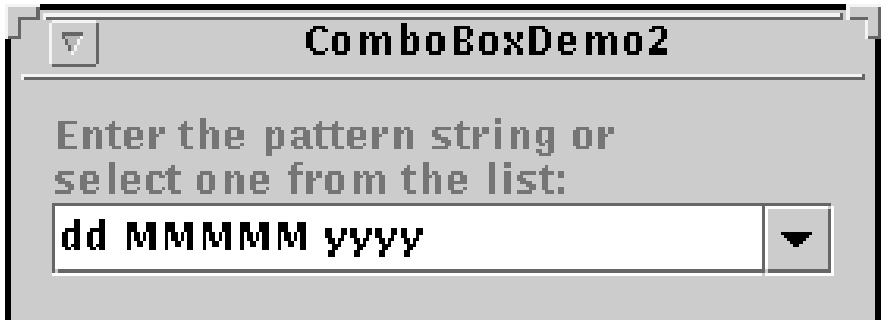
Les boutons

- Java propose différent type de boutons:
  - Le bouton classique est un **JBouton**.
  - **JCheckBox** pour les case à cocher
  - **JRadioBouton** pour un ensemble de bouton
  - **JToggleButton** Super Classe de CheckBox et Radio
  - …

Interface graphique (GUI)

JComboBox

- Un **JComboBox** est un composant permettant de faire un choix parmi plusieurs propositions.

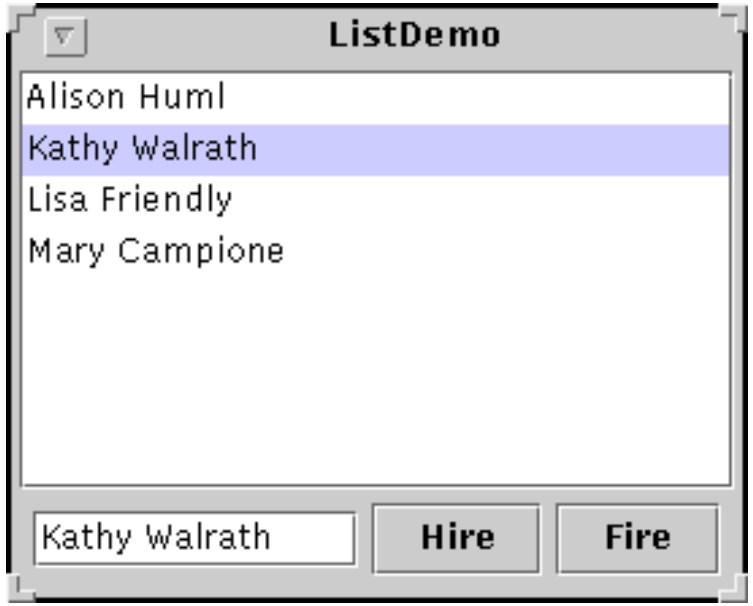


- Quelques méthodes:
  - **public JComboBox(Vector v)**
  - **public JComboBox(ComboBoxModel c)**
  - **Object getSelectedItem()**
  - **void addItem(Object o)**

Interface graphique (GUI)

JList

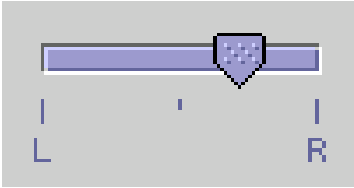
- Une JList propose plusieurs éléments rangés en colonne.
- Une JList peut proposer une sélection simple ou multiple
- Les JList sont souvent contenues dans un scrolled pane
- Quelques méthodes:
  - **public JList(Vector v)**
  - **public JList(ListModel l)**
  - **Object[] getSelectedValues()**



Interface graphique (GUI)

JSlider

- Les ***JSlider*** permettent la saisie graphique d'un nombre
- Un JSlider doit contenir les bornes max et min

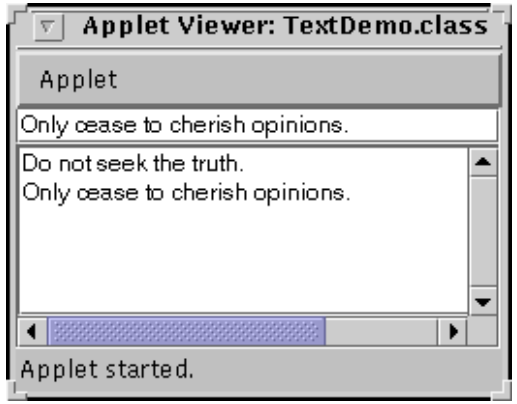


- Quelques méthodes:
  - **public JSlider(int min ,int max, int value)**
  - **public void setLabelTable(Dictionary d)**

Interface graphique (GUI)

JTextField

- Un ***JTextField*** est un composant qui permet d'écrire du texte.
- Un JTextField a une seul ligne contrairement au ***JTextArea***
- Le ***JPasswordField*** permet de cacher ce qui est écrit

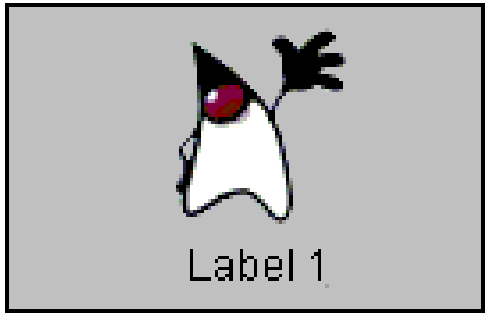


- Quelques méthodes:
  - ***public JTextField(String s)***
  - ***public String getText()***

Interface graphique (GUI)

JLabel

- Un ***JLabel*** permet d'afficher du texte ou une image.
- Un JLabel peut contenir plusieurs lignes et il comprend les tag HTML.



- Quelques méthodes:
  - **public JLabel(String s)**
  - **public JLabel(Icon i)**

Menu

- Si on a une barre de menu ***JMenuBar***, on ajoute des ***JMenu*** dans la barre.

- Exemple:

```
JMenuBar menuBar = new JMenuBar();
setJMenuBar(menuBar);
JMenu menu = new JMenu("A Menu");
menuBar.add(menu);
JMenuItem menuItem = new JMenuItem("menu item");
menu.add(menuItem);
```

Interface graphique (GUI)

JTree

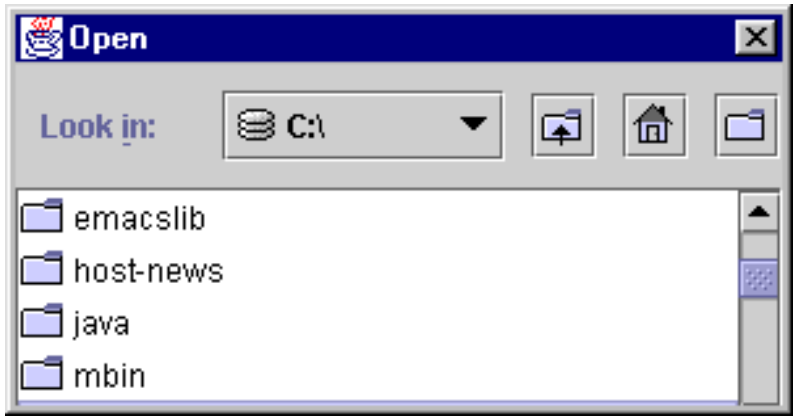
- Un ***JTree*** permet d'afficher des informations sous forme d'arbre. Les nœuds de l'arbre sont des objets qui doivent implanter l'interface MutableTreeNode. Une classe de base est proposée pour les nœuds : DefaultMutableTreeNode. Pour construire un arbre il est conseillé de passer par la classe TreeModel qui est la représentation abstraite de l'arbre.
- Pour construire un arbre:

```
rootNode = new DefaultMutableTreeNode("Root");
treeModel = new DefaultTreeModel(rootNode);
tree = new JTree(treeModel);
childNode = new DefaultMutableTreeNode ("Child");
rootNode.add(childNode);
```

Interface graphique (GUI)

JFileChooser

- Un ***JFileChooser*** permet de sélectionner un fichier en parcourant l'arborescence du système de fichier.

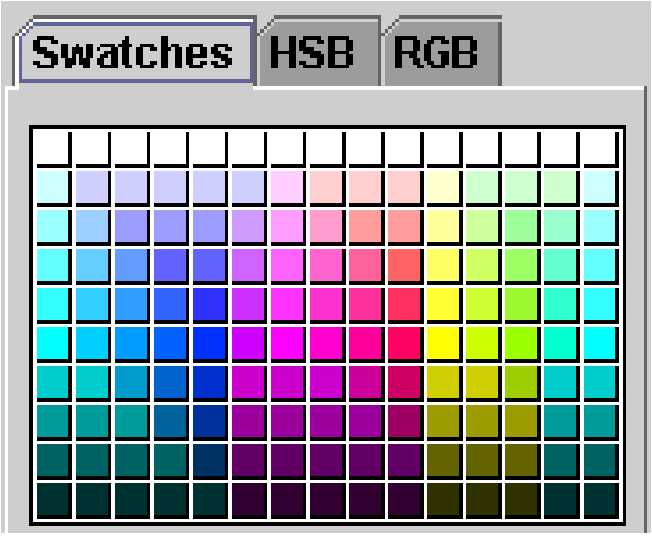


- Exemple :

```
JFileChooser fc = new JFileChooser();
int returnVal = fc.showOpenDialog(aFrame);
if (returnVal == JFileChooser.APPROVE_OPTION) {
        File file = fc.getSelectedFile();
}
```

JColorChooser

- Un ***JColorChooser*** permet de choisir une couleur

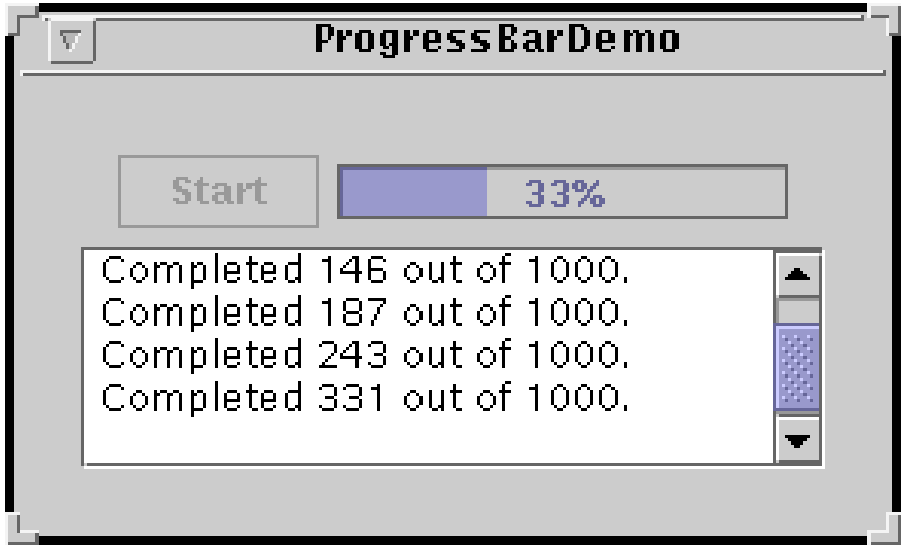


- Une méthode :

public static Color showDialog(Component c , String title , Color initialColor);

JProgressBar

- Un JProgressBar permet d'afficher une barre de progression.



- Quelques méthodes :

  public JProgressBar();

  public JProgressBar(int min, int max);

  public void setValue(int i);

Interface graphique (GUI)

Utilisation d'un composant de GUI

1. Création
   - Instancier un objet: **b = new JButton("press me");**
2. Configuration
   - Propriétés: **b.text = "press me";** //évité en java
   - Methods: **b.setText("press me");**

- L'ajout à un panneau

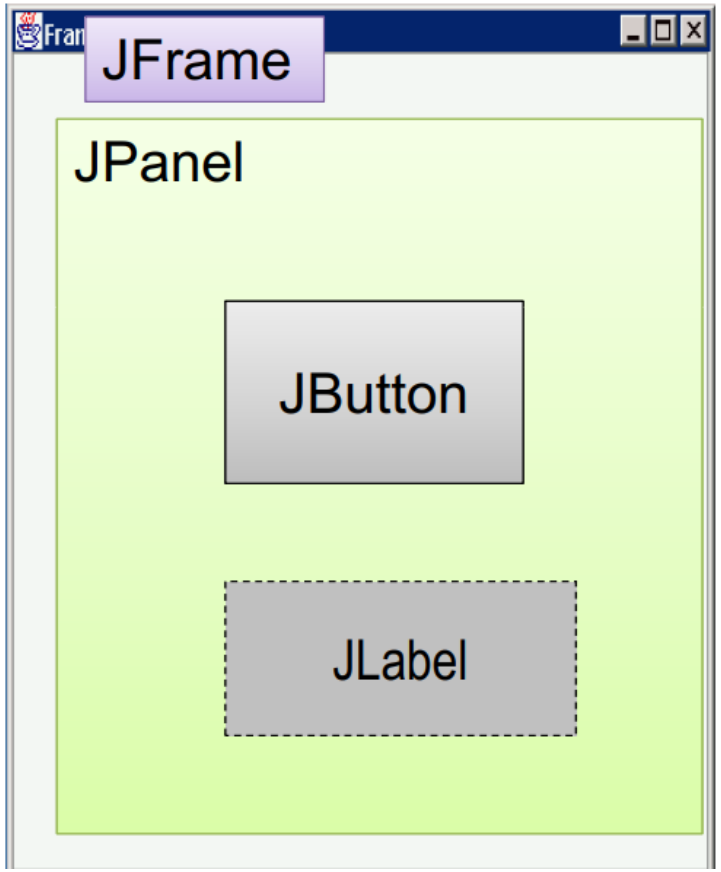
  panel.add(b);

- L'observation (écoute-lui)
  - Evénements: écouteurs (Listeners)

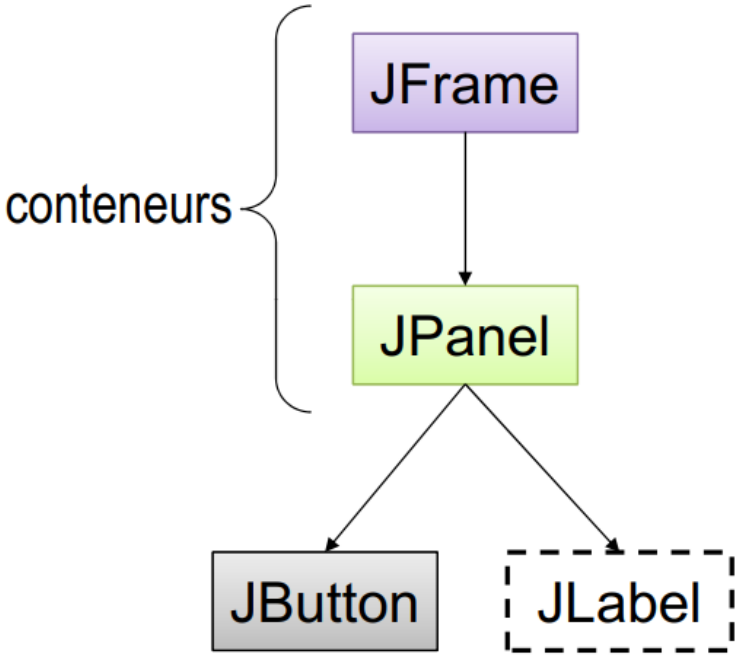
Interface graphique (GUI)

Anatomie d'une application avec GUI

GUI



Structure interne



Interface graphique (GUI)

Utilisation d’un composant de GUI

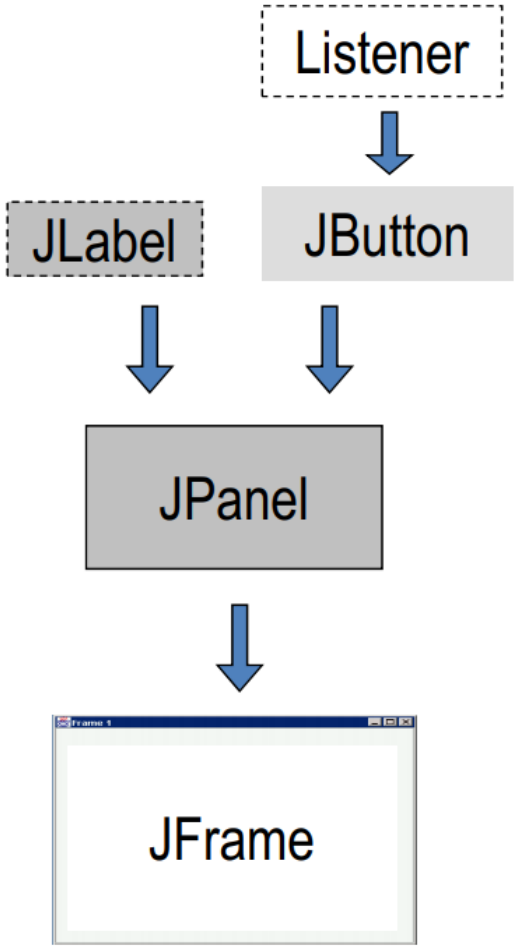
1. Crée-le
2. Configure-le
3. Ajoute-lui ses fils (si conteneur)
4. Ajoute-le à son parent (si pas JFrame)
5. Écoute-lui

L’ordre est important

Interface graphique (GUI)

Utilisation d'un composant de GUI

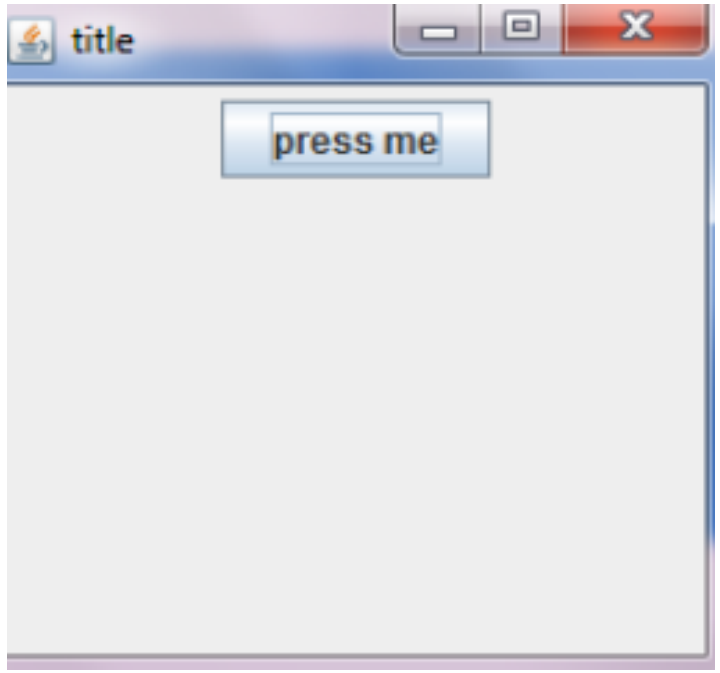
1. La création:
    1. Fenêtre (Frame)
    2. Panneau (Panel)
    3. composants (Components)
    4. Écouteurs (Listeners)
2. L'ajout: (bottom up)
    1. Écouteurs aux composants
    2. composants au panneau
    3. panneau à la fenêtre



Interface graphique (GUI)

Code de l'application

```
public class hello {
  public static void main(String[] args){
        JFrame f = new JFrame("title");
        JPanel p = new JPanel( );
        JButton b = new JButton("press me");
        p.add(b); // add button to panel
        f.setContentPane(p); // add panel to frame
        f. setVisible(true);
    }
}
```



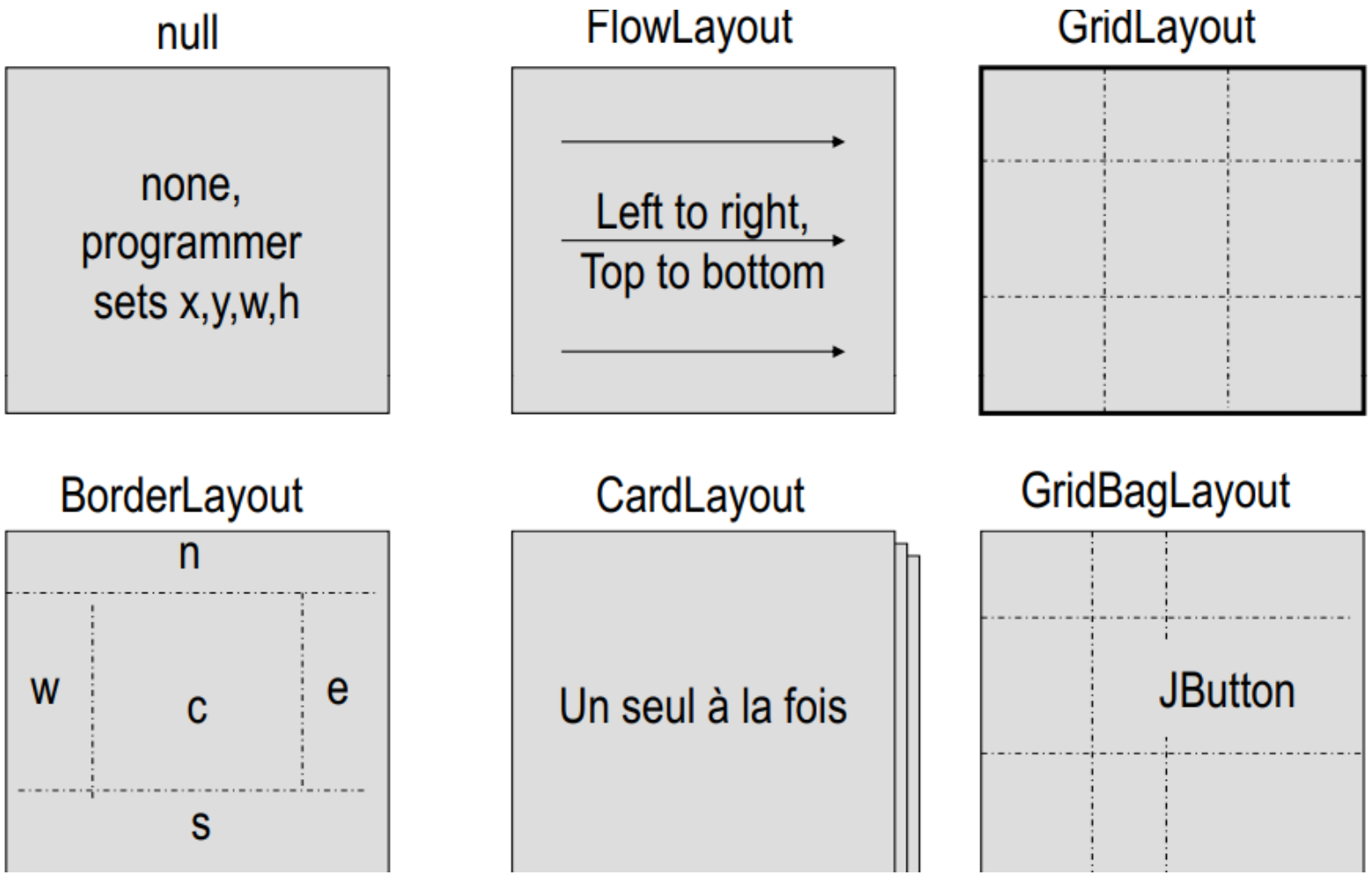
Interface graphique (GUI)

Positionnement des composants : Architecture de Layout

- Pour placer des composants dans un conteneur, Java propose une technique de **Layout**.
- Un **Layout** est une entité Java qui place les composants les uns par rapport aux autres.
- Le **Layout** s'occupe aussi de réorganiser les composants lorsque la taille du container varie.
- Il y a plusieurs **Layout** : ***BorderLayout, BoxLayout, CardLayout, FlowLayout, GridLayout, GridBagLayout.***
- Un **Layout** n'est pas contenu dans un conteneur, il gère le positionnement.

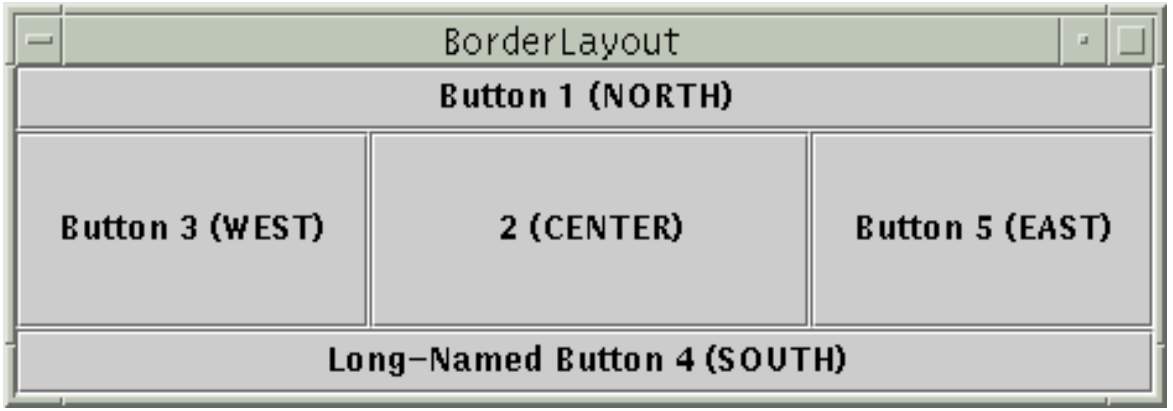
Interface graphique (GUI)

Heuristiques du gestionnaire de disposition



Positionnement des composants : BorderLayout

- Le ***BorderLayout*** sépare un conteneur en cinq zones: **NORTH, SOUTH, EAST, WEST** et **CENTER.**
- Lorsque l'on agrandit le conteneur, le centre s'agrandit. Les autres zone prennent uniquement l'espace qui leur est nécessaire.



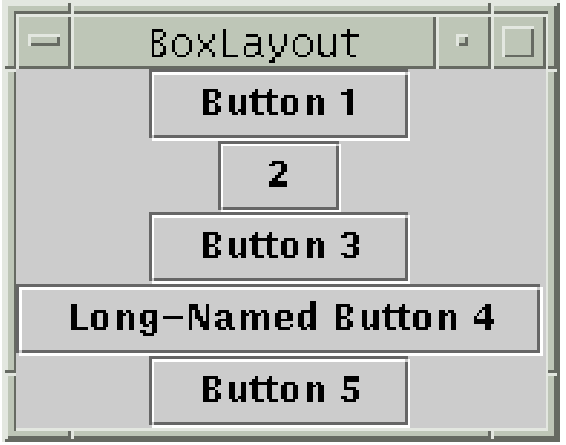
- Exemple :

```
Container contentPane = getContentPane();
contentPane.setLayout(new BorderLayout());
contentPane.add(new JButton("Button 1 (NORTH)"), BorderLayout.NORTH);
```

Interface graphique (GUI)

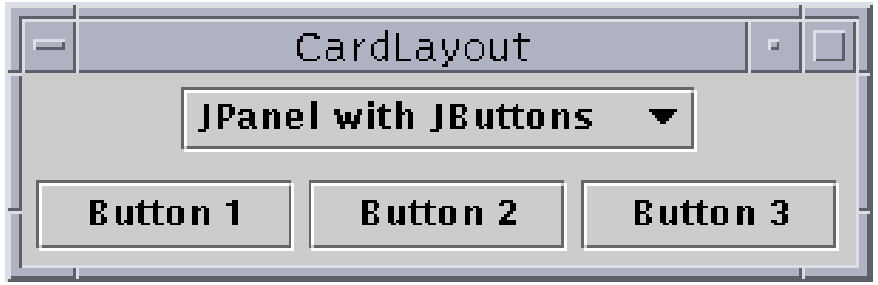
Positionnement des composants : BoxLayout

- Un ***BoxLayout*** permet d'empiler les composants du conteneur (soit de verticalement, soit horizontalement)
- Ce ***Layout*** essaye de donner à chaque composant la place qu'il demande
- Il est possible de rajouter des blocs invisible.
- Il est possible de spécifier l'alignement des composants (centre, gauche, droite)



Positionnement des composants : CardLayout

- Un **CardLayout** permet d'avoir plusieurs conteneurs ; les uns au dessus des autres (comme un jeux de cartes).



- Exemple :

```
JPanel cards;
final String BUTTONPANEL = "JPanel with JButtons";
final String TEXTPANEL = "JPanel with JTextField";
cards = new JPanel();
cards.setLayout(new CardLayout());
cards.add(p1,BUTTONPANEL);
cards.add(p2,TEXTPANEL);
```

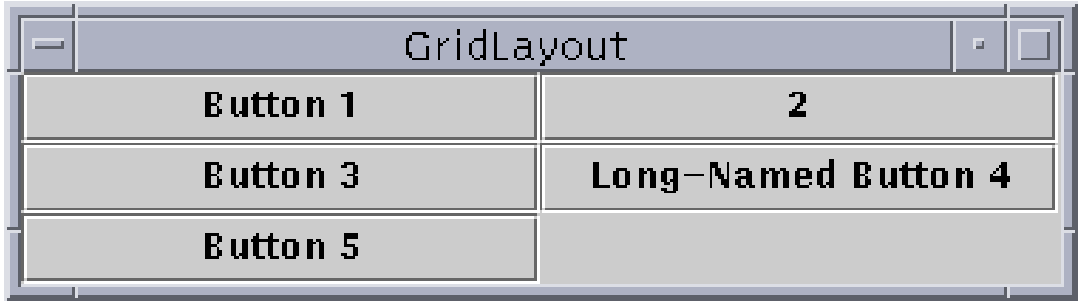
Positionnement des composants : FlowLayout

- Un **FlowLayout** permet de ranger les composants dans une ligne. Si l'espace est trop petit, une autre ligne est créée
- Le FlowLayout est le layout par défaut des JPanel
- Exemple :

```
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
contentPane.add(new JButton("Button 1"));
contentPane.add(new JButton("2"));
contentPane.add(new JButton("Button 3"));
contentPane.add(new JButton("Long-Named Button 4"));
contentPane.add(new JButton("Button 5"));
```

Positionnement des composants : GridLayout

- Un **GridLayout** permet de positionner les composants sur une grille.



- Exemple :

```
Container contentPane = getContentPane();
contentPane.setLayout(new GridLayout(3,2));
contentPane.add(new JButton("Button 1"));
contentPane.add(new JButton("2"));
contentPane.add(new JButton("Button 3"));
contentPane.add(new JButton("Long-Named Button 4"));
contentPane.add(new JButton("Button 5"));
```

Positionnement des composants : GridBagLayout

- Le **GridBagLayout** est le layout le plus complexe. Il place les composants sur une grille, mais des composants peuvent être contenus dans plusieurs cases. Pour exprimer les propriétés des composants dans la grille, on utilise un GridBagConstraints.
  - Un GridBasConstraints possède :
    - gridx, gridy pour spécifier la position
    - gridwidth, gridheight pour spécifier la place
    - fill pour savoir comment se fait le remplissage
  - …
- Exemple :

```
GridBagLayout gridbag = new GridBagLayout();
GridBagConstraints c = new GridBagConstraints();
JPanel pane = new JPanel();
pane.setLayout(gridbag);
gridbag.setConstraints(theComponent, c);
pane.add(theComponent);
```

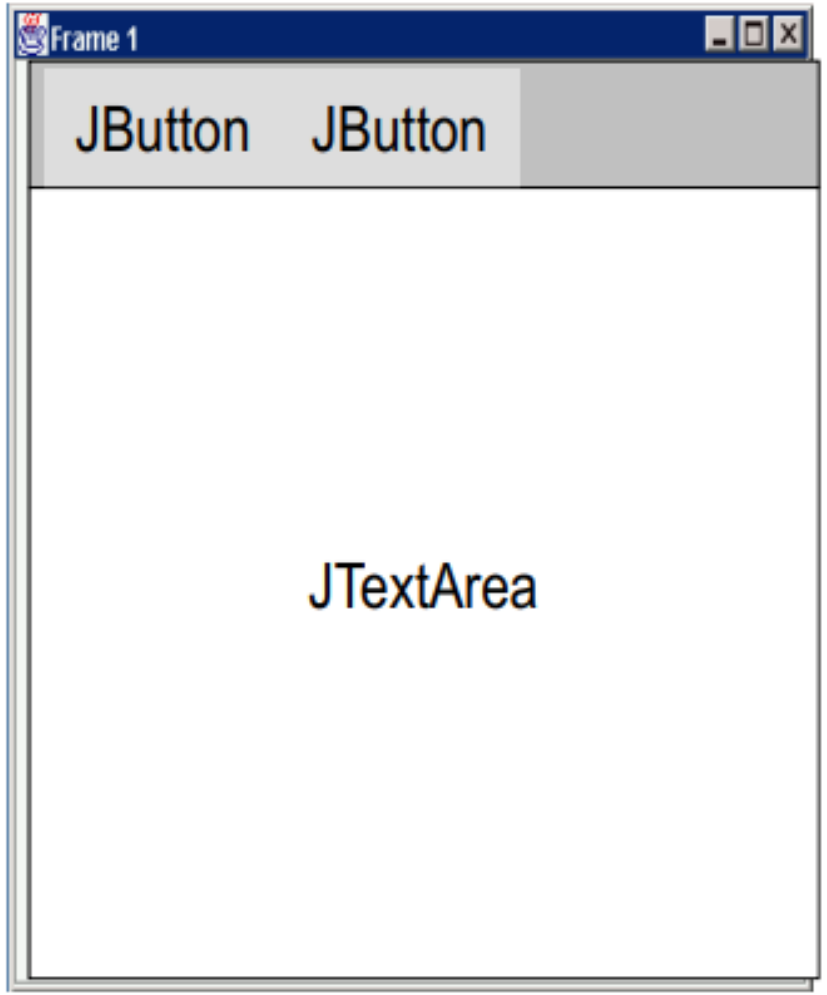
Interface graphique (GUI)

Création de Layout

- Il est possible de construire son propre Layout
- Un layout doit implanter l'interface java.awt.LayoutManager ou java.awt.LayoutManager2

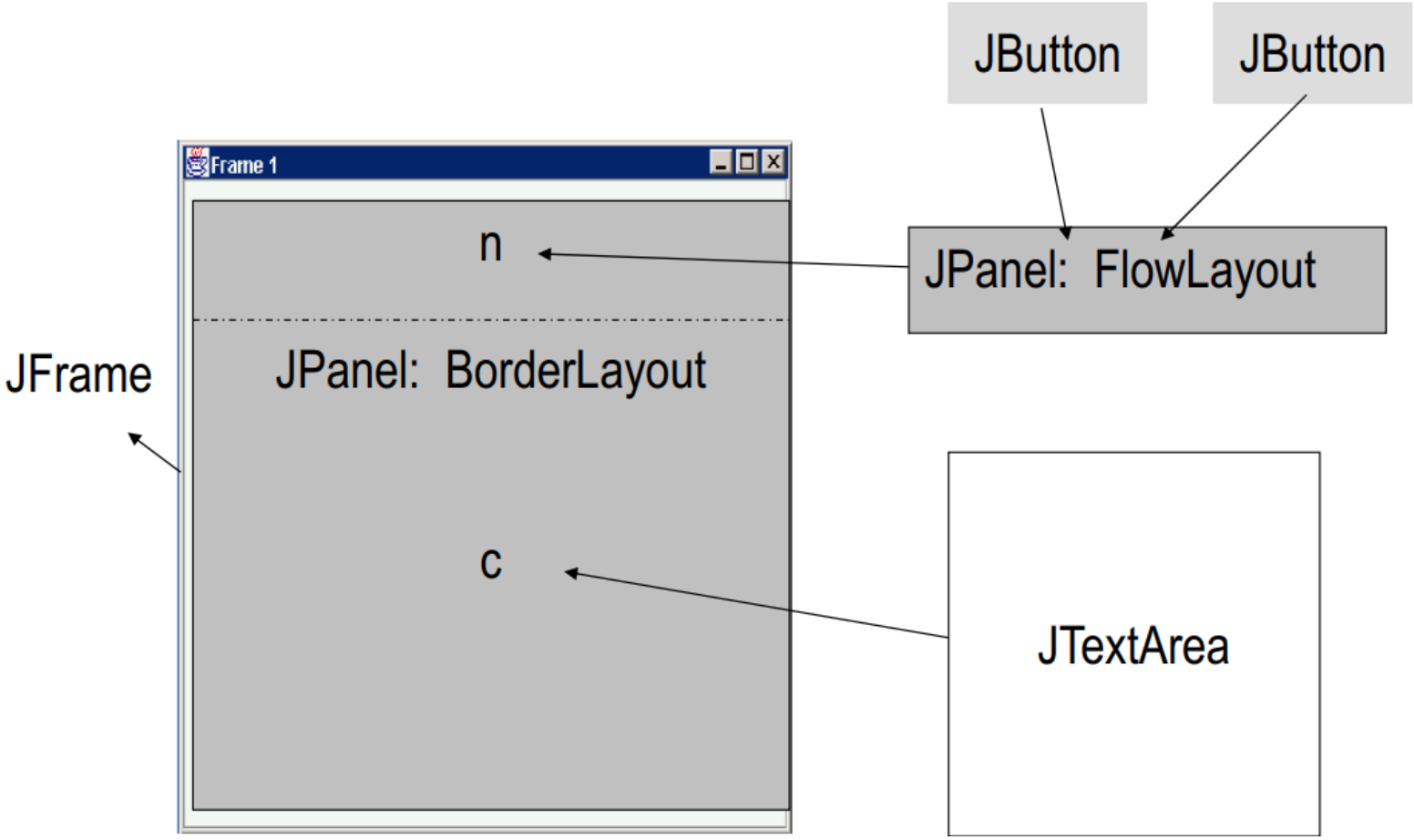
Interface graphique (GUI)

Combinaisons



Interface graphique (GUI)

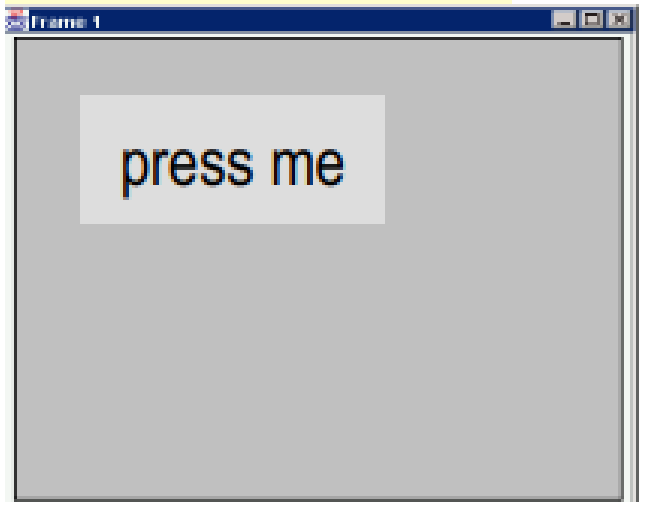
Combinaisons



Interface graphique (GUI)

Code :

```
JFrame f = new JFrame("title");
JPanel p = new JPanel();
JButton b = new JButton("press me");
b.setBounds(new Rectangle(10,10, 100,50));
p.setLayout(null);
p.add(b); // add button to panel
f.setContentPane(p); // add panel to frame
f.setVisible(true);
```

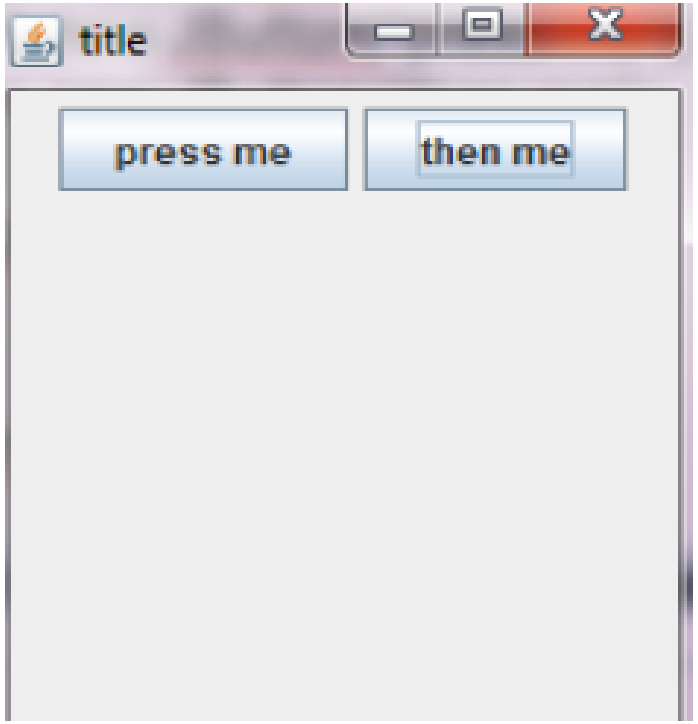


Interface graphique (GUI)

Code :

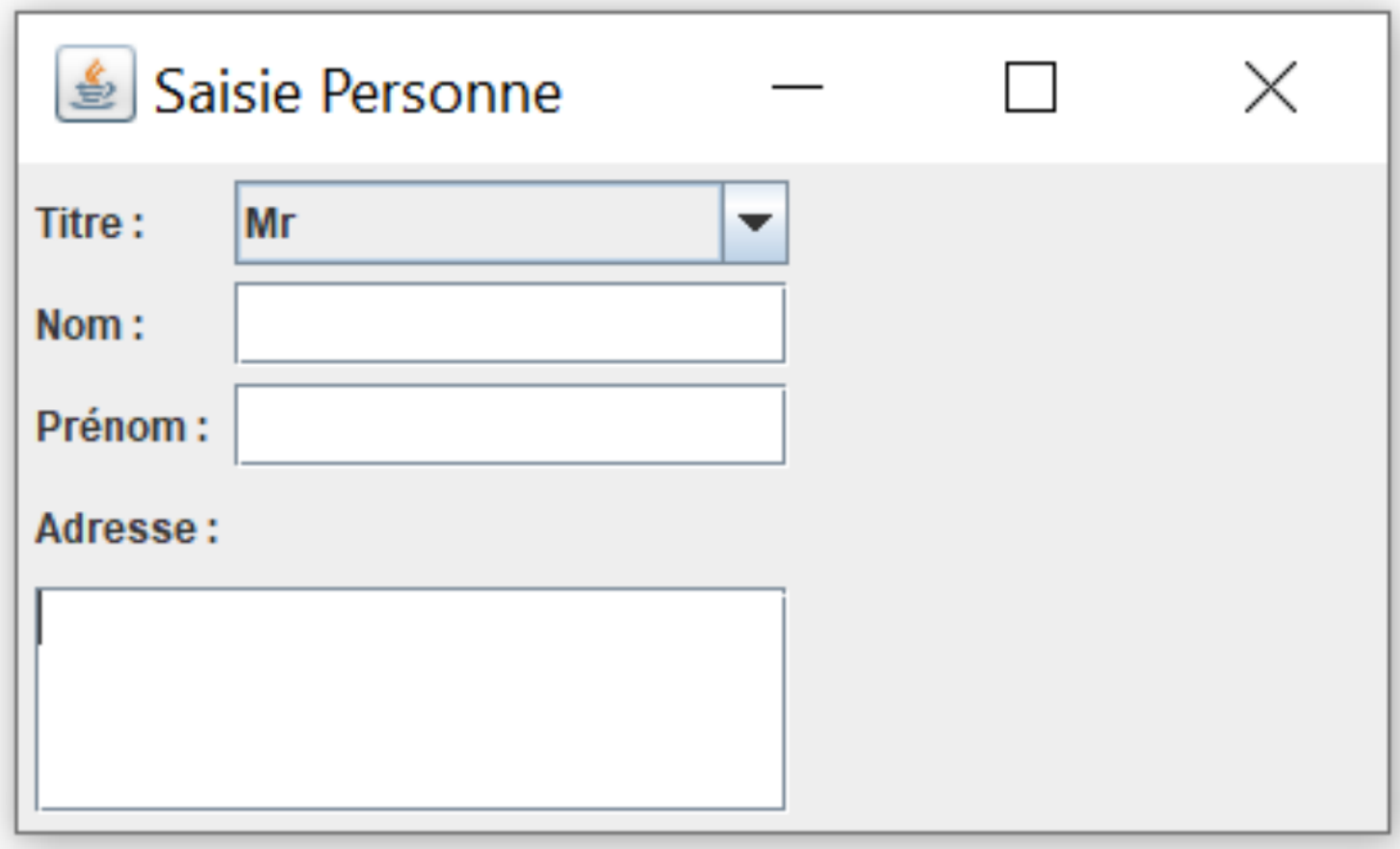
Affecter le layout managerr avant l'ajout des components

```
JFrame f = new JFrame("title");
JPanel p = new JPanel();
JButton b = new JButton("press me");
FlowLayout L = new FlowLayout( );
JButton b1 = new JButton("press me");
JButton b2 = new JButton("then me");
p.setLayout(L);
p.add(b1);
p.add(b2);
f.setContentPane(p);
f.setVisible(true);
```



Interface graphique (GUI)

Exemple :



Interface graphique (GUI)

Exemple : code

```
import java.awt.*;
import javax.swing.*;
public class PanneauPersonne extends JPanel {
private final static String[] TITRES = {"Mr", "Mme", "Melle" };
public PanneauPersonne()
{// Création d'un panneau pour les labels de 4 lignes et 1 colonne
JPanel panneauLabels = new JPanel(new GridLayout(4,1,5,5));
panneauLabels.add(new JLabel("Titre :"));
panneauLabels.add(new JLabel("Nom :"));
panneauLabels.add(new JLabel("Prénom :"));
panneauLabels.add(new JLabel("Adresse :"));
```

Interface graphique (GUI)

Exemple : code (suite)

```
// Création d'un panneau pour les composants de saisie de 4
//lignes et 1 colonne
JPanel panneauSaisie = new JPanel( new GridLayout(4,1,5,5));
panneauSaisie.add(new JComboBox(TITRES));
panneauSaisie.add(new JTextField(10));
panneauSaisie.add(new JTextField(10));
// Utilisation sur le panneau courant du layout BorderLayout
// avec un écart de 5 pixels
setLayout(new BorderLayout(5,5));
add(panneauLabels,BorderLayout.WEST);
add(panneauSaisie,BorderLayout.CENTER);
add(new JScrollPane(new JTextArea(4,20)), BorderLayout.SOUTH);
} }
```

Interface graphique (GUI)

Exemple : code (suite)

```
import java.awt.*;
import javax.swing.*;
public class ExempleEvolue {
public static void main(String[] args) {
// Création de la fenêtre (javax.swing.JFrame).
JFrame fenetre = new JFrame("Saisie Personne");
// Récupération du conteneur de la fenêtre (java.awt.Container).
Container panneau = fenetre.getContentPane();
panneau.setLayout(new FlowLayout(FlowLayout.LEFT,5,5));
```

Interface graphique (GUI)

Exemple : code (suite)

```
// Création du panneau de saisie de la personne
PanneauPersonne lePanneauPersonne = new PanneauPersonne();
panneau.add(lePanneauPersonne);
// Calcul de la taille préférée de la fenêtre en fonction de son contenu.
fenetre.pack();
// Changement de l'opération de fermeture en fonction de son contenu.
fenetre.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// Affichage de la fenêtre.
fenetre.setVisible(true);
}
}
```

Interface graphique (GUI)

Atelier : Création d'une calculatrice simple

Créer une interface graphique pour une calculatrice simple qui peut effectuer des opérations de base tels que l'addition, la soustraction, la multiplication et la division.

Spécifications :

1. Créez une fenêtre JFrame pour l'interface graphique de la calculatrice.
2. Ajoutez des composants Swing tels que des boutons et des zones de texte pour les nombres et le résultat.
3. Concevez le placement et la disposition des composants pour créer une interface utilisateur conviviale pour une calculatrice.
4. Implémentez la logique pour effectuer les opérations de base (addition, soustraction, multiplication et division) en fonction des valeurs entrées par l'utilisateur.
5. Affichez le résultat dans la zone de texte appropriée.

Royaume du Maroc
Université Cadi Ayyad
Faculté des Sciences Semlalia
Marrakech

Programmation Orientée objet Java

Pr. *Ilyass OUAZZANI TAYBI*

E-mail : i.ouazzani@uca.ac.ma

Département : Informatique FSSM

Université Cadi Ayyad

Licence ISI S5

Année universitaire : 2025/2026